EXERCICE 07 · CHANGEMENTS ENTRE LES ÉTAPES

La maman et l'enfant

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `- ligne retirée` `@@ position dans le fichier`

1. **Point de départ** · template (5 fichiers, en entier)
2. **Étape 1 → 2** · correction (2 fichiers)

Départ Point de départ : template

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (137 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

Images utilisées (non affichées) : public/avatars/enfant.svg, public/avatars/maman.svg

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 16 lignes

```jsx
import "./App.css";
import Parent from "./components/Parent";

function App() {
  return (
    <div>
      <div className="header">
        <h1>La maman et l'enfant</h1>
        <p>La maman donne un ordre, l'enfant répond.</p>
      </div>
      <Parent />
    </div>
  );
}

export default App;
```

DÉPART **src/components/Parent.jsx** 33 lignes

```jsx
import Child from "./Child";

// Exercice 7 · template : l'interface est prête, à vous d'ajouter la logique (les TODO)
function Parent() {
  // TODO 1 : créer deux states
  //   - le message de l'ordre (au départ "Hello from Parent")
  //   - le statut de l'ordre (au départ "")

  // TODO 2 : créer une fonction getStatus() qui renvoie "Accepted", "Denied" ou "Pending"
```

```jsx

  return (
    <div className="family">
      <div className="card">
        <img className="avatar" src="/avatars/maman.svg" alt="La maman" />
        <h2>Maman</h2>
        {/* TODO 3 : afficher le message du state à la place du texte en dur */}
        <p className="bubble">Hello from Parent</p>
        {/* TODO 4 : au clic, le message devient "Go to your room !" */}
        <button className="button button--primary">Send Order</button>
        <div>
          {/* TODO 7 : afficher le vrai statut avec getStatus() */}
          {/* Bonus : changer la classe status--pending selon le statut (status--accepted, status--denied) */}
          <p className="status status--pending">Order Status: Pending</p>
        </div>
      </div>

      {/* TODO 5 : passer à Child le prénom "Samir" et la fonction setOrderStatus */}
      <Child />
    </div>
  );
}

export default Parent;
```

DÉPART **src/components/Child.jsx** 18 lignes

```jsx
// TODO 5 : récupérer name et setOrderStatus dans les props
function Child() {
  return (
    <div className="card card--child">
      <img className="avatar" src="/avatars/enfant.svg" alt="L'enfant" />
      <h2>Enfant</h2>
      {/* TODO 5 : afficher le prénom reçu en props à la place de "Samir" */}
      <p className="bubble">Hello Samir, I am your Child</p>
      <div className="actions">
        {/* TODO 6 : au clic, mettre le statut à "accepted" ou à "denied" */}
        <button className="button button--accept">Accepter</button>
        <button className="button button--deny">Deny</button>
      </div>
    </div>
  );
}

export default Child;
```

DÉPART **index.html** 13 lignes

```html
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>La maman et l'enfant</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Étape 1 → 2 template → correction

Les deux states dans le Parent (message et statut), getStatus(), et l'enfant qui reçoit name et setOrderStatus en props pour répondre.

MODIFIÉ `src/components/Child.jsx` +10 −7

```
@@ -1,15 +1,18 @@
- // TODO 5 : récupérer name et setOrderStatus dans les props
- function Child() {
+ // L'enfant reçoit son prénom et la fonction du parent dans les props
+ function Child({ name, setOrderStatus }) {
    return (
      <div className="card card--child">
        <img className="avatar" src="/avatars/enfant.svg" alt="L'enfant" />
        <h2>Enfant</h2>
-       {/* TODO 5 : afficher le prénom reçu en props à la place de "Samir" */}
-       <p className="bubble">Hello Samir, I am your Child</p>
+       <p className="bubble">{`Hello ${name}, I am your Child`}</p>
        <div className="actions">
-         {/* TODO 6 : au clic, mettre le statut à "accepted" ou à "denied" */}
-         <button className="button button--accept">Accepter</button>
-         <button className="button button--deny">Deny</button>
+         {/* () => : la fonction est appelée au clic, pas tout de suite */}
+         <button className="button button--accept" onClick={() =>
setOrderStatus("accepted")}>
+           Accepter
+         </button>
+         <button className="button button--deny" onClick={() => setOrderStatus("denied")}>
+           Deny
+         </button>
        </div>
      </div>
    );
```

MODIFIÉ `src/components/Parent.jsx` +22 −14

```
@@ -1,31 +1,39 @@
+ import { useState } from "react";
  import Child from "./Child";

- // Exercice 7 · template : l'interface est prête, à vous d'ajouter la logique (les TODO)
+ // Exercice 7 · correction : le parent garde les states, l'enfant reçoit une fonction pour
remonter sa réponse
  function Parent() {
-   // TODO 1 : créer deux states
-   //   - le message de l'ordre (au départ "Hello from Parent")
-   //   - le statut de l'ordre (au départ "")
+   const [parentMessage, setParentMessage] = useState("Hello from Parent");
+   const [orderStatus, setOrderStatus] = useState("");

-   // TODO 2 : créer une fonction getStatus() qui renvoie "Accepted", "Denied" ou "Pending"
+   const getStatus = () => {
+     if (orderStatus === "accepted") return "Accepted";
+     if (orderStatus === "denied") return "Denied";
+     return "Pending";
+   };

    return (
      <div className="family">
        <div className="card">
          <img className="avatar" src="/avatars/maman.svg" alt="La maman" />
          <h2>Maman</h2>
```

```
16      -         {/* TODO 3 : afficher le message du state à la place du texte en dur */}
17      -         <p className="bubble">Hello from Parent</p>
18      -         {/* TODO 4 : au clic, le message devient "Go to your room !" */}
19      -         <button className="button button--primary">Send Order</button>
        20      +         <p className="bubble">{parentMessage}</p>
        21      +         <button
        22      +           className="button button--primary"
        23      +           onClick={() => setParentMessage("Go to your room !")}
        24      +         >
        25      +           Send Order
        26      +         </button>
20      27              <div>
21      -           {/* TODO 7 : afficher le vrai statut avec getStatus() */}
22      -           {/* Bonus : changer la classe status--pending selon le statut (status--accepted, status--denied) */}
23      -           <p className="status status--pending">Order Status: Pending</p>
        28      +           {/* Bonus : la classe change avec le statut (status--pending, status--accepted, status--denied) */}
        29      +           <p className={`status status--${getStatus().toLowerCase()}`}>
        30      +             {`Order Status: ${getStatus()}`}
        31      +           </p>
24      32            </div>
25      33          </div>
26      34
27      -       {/* TODO 5 : passer à Child le prénom "Samir" et la fonction setOrderStatus */}
28      -       <Child />
        35      +       {/* On passe le prénom ET la fonction setOrderStatus : c'est l'enfant qui va l'appeler */}
        36      +       <Child name={"Samir"} setOrderStatus={setOrderStatus} />
29      37        </div>
30      38      );
31      39    }
```